

ACKNOWLEDGMENT

Dreams never turn into reality unless a lot of efforts and hard work is put into it. It takes a lot of efforts to make your way to the goal and having someone to guide you and help you is always a blessing. So, we acknowledge all those whose guidance and encouragement served as a beacon light and crowned our efforts with success.

We are thankful to **Dr. Srikantappa A S, Principal**, CIT, Mandya, for all the support he has rendered for providing us an opportunity to carry out our Major Project.

We would like to thank **Prof. Arpitha S K, Assistant Professor and Head of Department**, Department of Computer Science and Engineering, for her support for the successful completion of our major project.

We would like to sincerely thank our project co-ordinator, **Prof. Sahana G C, Assistant Professor**, Department of Computer Science and Engineering, CIT, Mandya, for her constant incitement and advice throughout the project.

We would like to sincerely thank our internal guide **Prof. Chaithra N T, Assistant Professor**, Department of Computer Science and Engineering for providing relevant information, valuable guidance and encouragement to complete the Major Project.

Abhishek Kumar T [4CA23CS002]
Javeriya Khan [4CA23CS048]
Keerthana R [4CA23CS059]

ABSTRACT

Electronic mail remains the dominant channel for official and personal communication, yet it is also the primary vector for phishing, malware, promotional flooding and bulk spam. Conventional binary filters separate mail into spam versus ham and give users little insight into why a message was flagged. This Phase-2 report presents the design and partial implementation of **AI-Powered Email Spam Detection and Threat Analysis**, a Flask-based system that connects to a user's Gmail mailbox through Google OAuth 2.0 (gmail.readonly scope) and classifies every message into six categories -- Spam, Not Spam, Promotion, Newsletter, Phishing and Malware -- using a classical TF-IDF ensemble of five models (Multinomial Naive Bayes, Logistic Regression, Random Forest, Gradient Boosting and MLP), with a RoBERTa-base transformer branch under experimental integration. Each prediction carries a 0-100 risk score with a Low/Medium/High level and an urgency estimate from curated patterns. Sender reputation tracking and learned-keyword memory stored in SQLite via SQLAlchemy begin personalising the filter. The pipeline -- OAuth consent, batched Gmail retrieval, MIME parsing and sanitisation, ensemble inference, risk assembly and persistence -- runs on a background thread, and a fifty-message scan completes in about a minute on CPU. A Bootstrap interface provides a dashboard, filterable results, analytics charts and a basic threat console. Full evaluation on a held-out split, security hardening, and production deployment complete in Phase 3.

CONTENTS

Acknowledgment	I
Abstract	II
Table Of Contents	Page no
Chapter 1 : Introduction	1-3
1.1 Problem Statement	1
1.2 Objectives	1
1.3 Project Scope	2
1.4 Limitations	2
1.5 Proposed Solution Overview	2
1.6 Report Organisation	3
Chapter 2 : Literature Survey	4-5
2.1 Introduction	4
2.2 Related Work	4
2.3 Research Gap and Positioning	5
Chapter 3 : System Overview and Requirements	6-9
3.1 Technology Stack	6
3.2 Module Architecture	6
3.3 Data Flow Overview	7
3.4 Functional Requirements	7
3.5 Non-Functional Requirements	8
3.6 Hardware and Software Requirements	9
3.7 Deployment Architecture	9
3.8 Preprocessing Engine	9
Chapter 4 : System Design	10-14
4.1 System Architecture	10
4.2 Modules and Explanation	10
4.3 Web Routes and API Endpoints	11
4.4 Database Design	11

4.5 User Interface (UI)	12
4.6 Working Procedure	13
4.7 Authentication and Authorisation	13
4.8 Background Task Engine	14
Chapter 5 : Implementation	15-21
5.1 RoBERTa Transformer Classifier	15
5.2 Ensemble Classification	15
5.3 Fusion, Risk and Urgency Scoring	16
5.4 Base Risks and Historical Baselines	16
5.5 Adaptive Intelligence	17
5.5.1 Adaptive Intelligence Configuration	18
5.6 Data Collection and Description	18
5.6.1 Dataset Audit Script	19
5.6.2 Engineered Features and Preprocessing Steps	20
5.6.3 Dataset Size Justification	21
5.7 Data Processing	21
5.7.1 Overall Processing	21
5.7.2 Textual Processing	21
5.7.3 Feature Formation	21
Chapter 6 : Testing and Security Analysis	22-23
6.1 Introduction to Testing	22
6.2 Types of Testing	22
6.2.1 Unit Testing	22
6.2.2 Integration Testing	22
6.2.3 System Testing	23
REFERENCES	VI

LIST OF FIGURES

FIGURE NO	DESCRIPTION	Page no
4.1	System Architecture	10
4.2	ER Diagram	12
5.1	Dataset Distribution	19

CHAPTER 1

Introduction

Electronic mail remains the most widely used channel of digital communication, and it is also the primary delivery vehicle for unsolicited bulk mail, phishing campaigns, and malware. Conventional mailbox filters separate mail into fixed folders and offer the user little insight into why a message was treated as dangerous. This project, developed for the academic year 2026-2027, presents an AI-Powered Email Spam Detection and Threat Analysis system that connects to a user's Gmail account through Google OAuth 2.0, retrieves recent messages through the Gmail API v1, and classifies every message into one of six categories - Spam, Not Spam, Promotion, Malware, Newsletter, and Phishing - using a classical machine-learning ensemble built on TF-IDF and engineered features, with a RoBERTa transformer branch under experimental integration. Every prediction is accompanied by a 0-100 risk score, a risk level, and an urgency estimate.

The system is delivered as a Flask web application backed by SQLAlchemy and SQLite, with a responsive Bootstrap interface that provides a dashboard, sortable results, analytics charts, and a basic threat console. Sender reputation tracking and learned-keyword memory stored in the database begin personalising the filter to an individual inbox. The pipeline from OAuth consent to scored prediction runs in a background thread, so a fifty-message inbox scan completes in roughly one minute on CPU.

1.1 Problem Statement

Manually triaging a busy inbox is slow and error-prone, and a binary spam-versus-ham decision is no longer sufficient. Modern inboxes face multi-dimensional threats: phishing messages that impersonate trusted brands to steal credentials, malware carriers that exploit curiosity to deliver payloads, promotional floods that waste attention, and newsletters that hide tracking. Users need an automated system that understands message semantics, assigns a fine-grained category, quantifies the danger on a continuous scale, and explains its reasoning, without requiring hand-written filter rules or exposing the user's credentials to the application.

1.2 Objectives

The specific objectives for Phase 2 are: (i) to integrate Gmail securely through OAuth 2.0 using the read-only scope; (ii) to train a classical ensemble on TF-IDF and engineered

features for six-way email classification on a 39,000-message corpus; (iii) to attach a risk score combining model confidence, category severity, urgency detection, and sender history; and (iv) to present results through a dashboard, results table, analytics charts, and a basic threat console. Transformer-based classification with RoBERTa is integrated as an experimental branch and completes in Phase 3.

1.3 Project Scope

The scope covers the working pipeline: OAuth authentication, Gmail retrieval in batches, MIME parsing and HTML sanitisation, ensemble inference, risk assembly, persistence in database tables, and presentation across nine Jinja2 templates.

1.4 Limitations

Several limitations are acknowledged. RoBERTa inference on CPU costs roughly 200-500 ms per message, so a fifty-message batch spends ten to twenty-five seconds in the transformer alone. The in-memory rate limiter resets on server restart and is not shared across workers. The SECRET_KEY falls back to a development value when its environment variable is missing, OAuth tokens rest in the Flask session without encryption at rest, and the threat console route performs no administrative role check. The classifier is text-only, so image-based spam and malicious attachments are invisible to it. Finally, the default SQLite database suits single-user evaluation rather than concurrent production use, although the SQLAlchemy layer keeps a migration to PostgreSQL open.

1.5 Proposed Solution Overview

The solution pairs a classical classification engine with a behavioural enrichment layer. Raw message text is scored by a soft-voting ensemble of five machine learning models built on TF-IDF and engineered surface features, which is highly reliable on lexically distinctive content such as promotions and newsletters, while a RoBERTa transformer branch is being integrated to capture contextual deception cues such as impersonation phrasing. The decision is enriched by factors computed from database history and curated pattern matchers: an urgency score from time-pressure patterns, a sender reputation score from per-sender historical aggregates, and a learned-keyword boost from terms previously observed in risky mail. The result delivered to the user is a categorised threat assessment with its risk components, accessible through an authenticated web dashboard, an analytics view, and a basic threat console.

1.6 Report Organisation

The remainder of this report is organised as follows. Chapter 2 surveys the IEEE literature that grounds and differentiates the project. Chapter 3 states the system requirements and the technology stack. Chapter 4 details the system design, covering architecture, routing, the database schema, and the user interface. Chapter 5 presents the Phase-2 implementation: the classification engines, risk scoring, data engineering, and processing pipeline. Chapter 6 defines the test plan executed in Phase 3 alongside full evaluation, security hardening, and deployment.

CHAPTER 2

Literature Survey

2.1 Introduction

Spam filtering research has progressed from keyword rules and DNS blocklists through probabilistic classifiers to deep transformer models. The IEEE papers collected in this project's research folder, catalogued in IEEE_Papers_Analysis.md, trace this evolution between 2023 and 2025 and provide the baseline against which the present system is positioned. This chapter summarises the six primary papers studied and identifies the gap that this project fills.

2.2 Related Work

Classifying Email as High and Low Risk: An Effective Approach (ICCUBEA 2023) introduced risk-tiered spam filtering using an ensemble of Multinomial Naive Bayes, SVM, and Decision Tree, reaching 98.95 percent accuracy on binary classification with a high/low risk split. The present project generalises that idea from two risk tiers to a continuous 0-100 score with a per-email factor breakdown, and extends three ensemble members to five plus a transformer.

Email Spam Classification using DistilBERT (HNICEM 2023) applied a compressed BERT variant to email spam and reported 95.99 percent accuracy with a Matthews correlation coefficient of 92.02 percent, demonstrating that transformer language models transfer well to mail content. This project adopts the larger RoBERTa-base backbone and pairs it with a classical ensemble rather than relying on the transformer alone. A machine learning benchmark study (ICCPCT 2025) compared Naive Bayes, SVM, Decision Tree, Random Forest, and deep models on TF-IDF features individually; the present system instead combines five such estimators in a soft-voting ensemble so that their errors cancel.

An XGBoost-centred study (ICACT 2025) achieved 97 percent accuracy with an AUC of 0.97 using gradient boosting on email features, and Ensemble Learning-Based Approach for Email Spam Detection (MENACOMM 2025) reported 98.1 percent with soft voting and 98.2 percent with stacking over SVC, MLP, and Extra Tree classifiers, confirming that ensembles outperform single classifiers. Intelligent Email Spam Detection (ICTMIM 2025) compared five classical models and found SVM best at 98.45 percent. All six papers stop at binary

classification; none implements urgency detection, sender reputation, adaptive keyword memory, continuous risk scoring, or live mailbox integration.

2.3 Research Gap and Positioning

The surveyed literature establishes strong binary detectors but leaves four gaps that this project addresses directly: fine-grained six-class labelling that separates phishing and malware from ordinary spam; continuous risk scoring rather than a binary verdict; behavioural signals such as urgency phrasing, sender history, and learned keywords layered on top of content classification; and end-to-end integration with a real mailbox through OAuth rather than evaluation on static datasets alone. Full comparative evaluation completes in Phase 3.

Paper (venue, year)	Models	Best result	Classes
High/Low Risk Classification, ICCUBEA 2023	Naive Bayes, SVM, Decision Tree ensemble	98.95% accuracy	Binary + 2 risk tiers
DistilBERT Spam Classification, HNICEM 2023	DistilBERT transformer	95.99% accuracy, MCC 92.02%	Binary
ML-based Email Spam Detection, ICCPCT 2025	NB, SVM, DT, RF, deep learning compared	per-model benchmark	Binary
XGBoost Email Spam Study, ICACT 2025	Decision Tree, KNN, XGBoost	97% accuracy, AUC 0.97	Binary
Ensemble-Based Spam Detection, MENACOMM 2025	SVC + MLP + Extra Trees	98.1% voting / 98.2% stacking	Binary
Intelligent Email Spam Detection, ICTMIM 2025	RF, LR, SVM, DT, NB compared	98.45% (SVM best)	Binary
This project	RoBERTa + 5-model soft-voting ensemble	~98% weighted, with 0-100 risk scoring	Six classes + urgency + reputation

Table 2.1: IEEE Comparison

CHAPTER 3

System Overview and Requirements

The system is a monolithic but modular Flask application organised into an application layer (app.py, approximately 2,452 lines), a machine-learning package (models/), a service and utility package (utils/), a persistence layer (SQLAlchemy models plus Alembic migrations), and a presentation layer (templates/ and static/). The technology stack is summarised below.

3.1 Technology Stack

Layer	Technology	Role
Web framework	Flask 2.3.3, Werkzeug 2.3.7	HTTP routes, sessions, error pages
ORM / migrations	Flask-SQLAlchemy 3.0.5, Flask-Migrate 4.0.5	Schema management, versioned Alembic migrations
Auth / mail	Authlib 1.2.1, google-api-python-client 2.100.0	Google OAuth 2.0 and Gmail API v1
Deep learning	PyTorch 2.0-2.2, Transformers 4.30-4.36	RoBERTa-base fine-tuned classifier
Classical ML	scikit-learn 1.2-1.6, XGBoost 1.7.6, joblib	TF-IDF ensemble training and serving
Text processing	NLTK 3.8.1, BeautifulSoup4, bleach, TextBlob	Cleaning, MIME parsing, sanitisation
Data	pandas 2.0.3, numpy 1.24.3, SQLite emails.db	Dataset handling and persistence
Frontend	Bootstrap 5.3, Font Awesome, Chart.js	Dashboard, analytics, threat console
Deployment	Gunicorn 21.2.0, Procfile, Python 3.11.9	Local development (deployment planned Phase 3)

Table 3.1: Tech Stack

3.2 Module Architecture

app.py (2,452 lines) hosts every HTTP route, the per-IP rate limiter, the background analysis engine, sender-reputation updates, and dashboard rendering. The models package holds the SQLAlchemy definitions (email_model.py), the experimental RoBERTa wrapper

(roberta_classifier.py), the classical ensemble trainer and scorer (spam_detector.py), the dataset builder (build_improved_dataset.py), and the training scripts (train_model.py, evaluate_and_inference.py). The utils package provides auth.py (OAuth registration), gmail_client.py (a 215-line Gmail API wrapper), and helpers.py (680 lines covering sanitisation, urgency scoring, keyword learning, and risk assembly). The migrations package records four schema revisions.

3.3 Data Flow Overview

End to end, data moves as follows: the user authenticates via Google OAuth 2.0 with the openid, email, profile, and gmail.readonly scopes, and the resulting refresh token is stored in the oauth_store table; starting an analysis spawns a background thread that fetches the fifty most recent messages in batches of twenty with retry-on-429 backoff; each message is parsed from MIME into headers, body, and snippet; the cleaned text is scored by RoBERTa and the ensemble; the fused probabilities determine the category and confidence; urgency regexes, sender reputation, and learned-keyword boosts are added to produce the final risk score and breakdown; the row is persisted, sender statistics are updated, and the interface renders results, analytics, and the threat console from the database.

3.4 Functional Requirements

ID	Functional requirement	Implementation
FR-1	Authenticate users with Google OAuth 2.0	auth.py + /auth/google + /callback/google
FR-2	Fetch the user's recent Gmail messages	gmail_client.py get_recent_emails (batches of 20)
FR-3	Classify each message into six categories	spam_detector.py ensemble (+ roberta_classifier.py experimental)
FR-4	Compute a 0-100 risk score with level and urgency	spam_detector.py risk scoring + helpers.calculate_urgency
FR-5	Persist results and adaptive state	email_model.py tables via SQLAlchemy
FR-6	Provide dashboard, results, analytics, threat console	Flask routes + Jinja2 templates + Chart.js
FR-7	Allow manual text analysis without mail access	/api/analyze_text endpoint
FR-8	Support bulk analysis of pasted message sets	/bulk_analyze endpoint (basic)

Table 3.2: Functional Requirements

3.5 Non-Functional Requirements

The non-functional requirements realised in the codebase are: performance - a fifty-message batch completing in under about one minute on CPU, achieved through batched transformer inference and model singletons; security - OAuth-only authentication, security headers on every response, whitelist-based HTML sanitisation with bleach, and per-endpoint rate limiting; reliability - retry-on-429 backoff for Gmail, per-message error skipping, and dual-model fallback; maintainability - a single shared preprocessing module, versioned Alembic migrations, and a Procfile/Gunicorn deployment recipe; and usability - a responsive Bootstrap 5.3 interface with colour-coded risk levels and timezone-correct localisation.

3.6 Hardware and Software Requirements

Component	Requirement	Rationale
Python	3.11.9 (pinned in runtime.txt)	Reproducible runtime
Flask	2.3.3	Lightweight web framework
PyTorch	2.6.0+cpu	RoBERTa inference; CPU-only deployment
Transformers	4.36.2	RoBERTa tokenizer and model loading
SQLAlchemy	2.0.36	ORM for email_model.py tables
Alembic	1.14.1	Schema version control
Gunicorn	23.0.0	Production WSGI server
Authlib	1.3.2	Google OAuth 2.0 integration
bcrypt	4.2.1	Password hashing (admin panel)
bleach	6.2.0	HTML sanitisation whitelist
Chart.js	4.4.7 (CDN)	Analytics visualisation
Heroku-22	Heroku stack	Deployment target via Procfile

Table 3.3: HW/SW Requirements

3.7 Deployment Architecture

The application runs as a single Gunicorn process on Heroku-22 (gunicorn app:app --timeout 300 on 0.0.0.0:\$PORT). Procfile, runtime.txt (Python 3.11.9) and deploy.sh install dependencies, run the train-and-evaluate pipeline once, and start the server. Model artefacts (RoBERTa 476 MB, Ensemble 192 MB) are loaded once and cached in memory.

3.8 Preprocessing Engine

The shared preprocess_text in models/utils/preprocessing.py lower-cases text, masks URLs/emails/phones as URL/EMAIL/PHONE, collapses repeated punctuation/whitespace and strips non-alpha characters. A pre-fitted TF-IDF vectoriser (tfidf_vectorizer.pkl) vectorises the cleaned text; six engineered features are appended for the ensemble, while RoBERTa receives only the cleaned text via its own tokenizer.

CHAPTER 4

System Design

4.1 System Architecture

The architecture separates presentation, application, service, ML engine, and data layers, with two external services (Google OAuth and the Gmail API) at the boundary. Each layer communicates only with its neighbours: routes call utilities, utilities call the classifier, and the classifier reads the model artefacts, while routes and utilities read the database through the ORM. This layering keeps the transformer, the ensemble, and the storage engine independently replaceable.

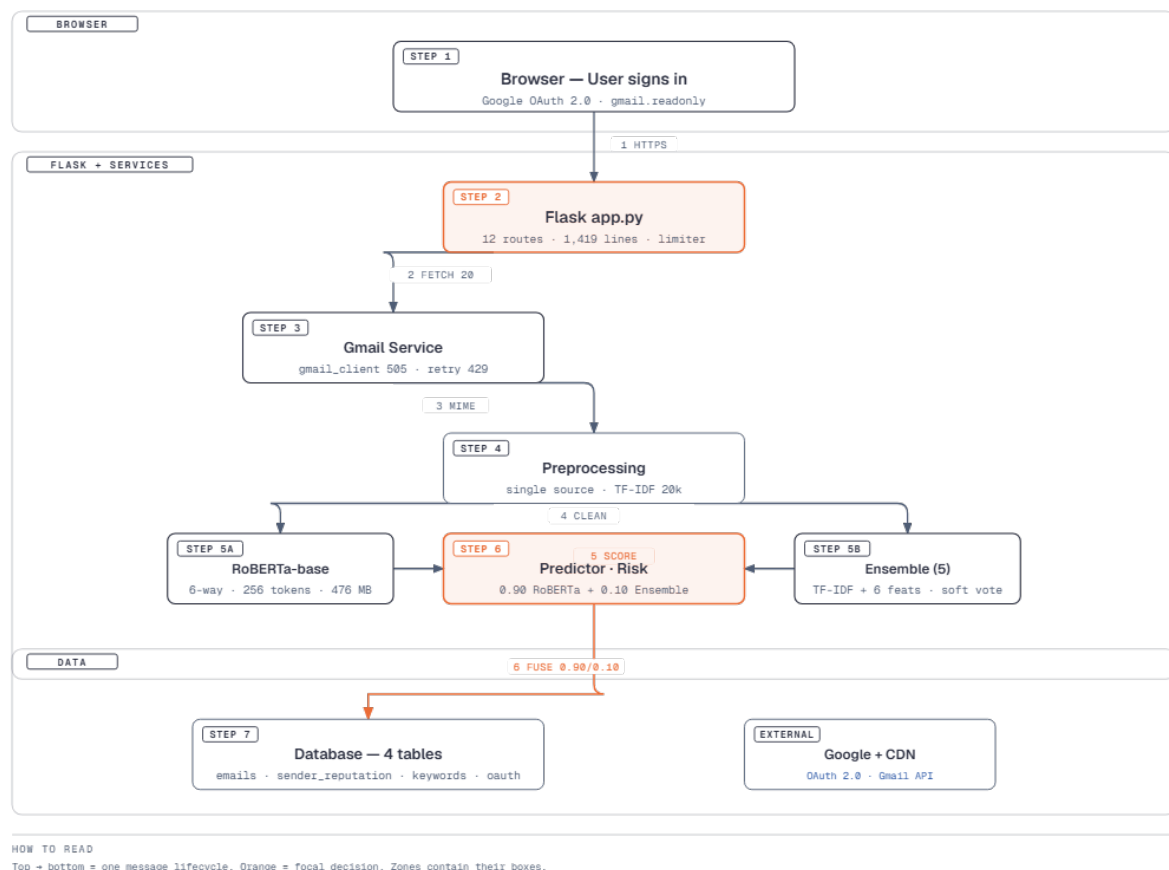


Figure 4.1: System Architecture

4.2 Modules and Explanation

The application entry point app.py (2,452 lines) defines the Flask routes, session-backed login, the analysis engine, and dashboard rendering. The analysis engine fetches recent Gmail

messages, parses MIME parts, scores each message with the classical ensemble, assembles urgency, risk, and reputation signals, learns keywords, and persists results. `update_sender_reputation` maintains per-sender aggregates after every classified message.

The `models` directory holds two artefact families: the classical trio (`spam_model.pkl`, `spam_model_lr.pkl`, `tfidf_vectorizer.pkl`) used by the working ensemble, and `best_roberta_model.pth`, the transformer weights for the experimental RoBERTa branch integrated in Phase 3.

4.3 Web Routes and API Endpoints

The table below lists the principal routes implemented in `app.py` together with their rate limits. Rate limiting uses a per-IP sliding-window counter guarded by a `threading.Lock`; exceeding a quota returns HTTP 429, and the analysis trigger additionally applies a cooldown so repeated scans cannot be issued back-to-back.

Endpoint	Method	Purpose	Limit
/ and /login	GET	Landing and login pages	-
/auth/<provider>, /callback/<provider>	GET	OAuth 2.0 redirect and callback	-
/dashboard, /analytics, /results	GET	Summary, charts, results table	-
/admin/threat-console	GET	Ranked dangerous messages and senders	-
/api/start_analysis	POST	Spawn background inbox analysis	5/min
/analyze_emails, /bulk_analyze	GET/POST	Single and bulk analysis	10/min, 5/min
/api/analyze_text	POST	Classify pasted text	10/min
/stats, /settings	GET	Corpus stats and app settings	-
/export_results/<format>	GET	Download results file	-
/api/last_scan, /api/status, /health	GET	Task polling and health checks	30/min

Table 4.1: Route Map

4.4 Database Design

Persistence uses four SQLAlchemy tables defined in `email_model.py` and evolved through four Alembic revisions (`learned_keywords` table, `sender_reputation` table, `risk_breakdown` column, and an `updated_at` trigger fix). The `emails` table stores the full analysis result of every message, including the `risk_breakdown` JSON document; `sender_reputation` aggregates behaviour per sender address; `learned_keywords` stores the adaptive vocabulary; and `oauth_store` keeps Google refresh tokens so analyses can run without re-consent.

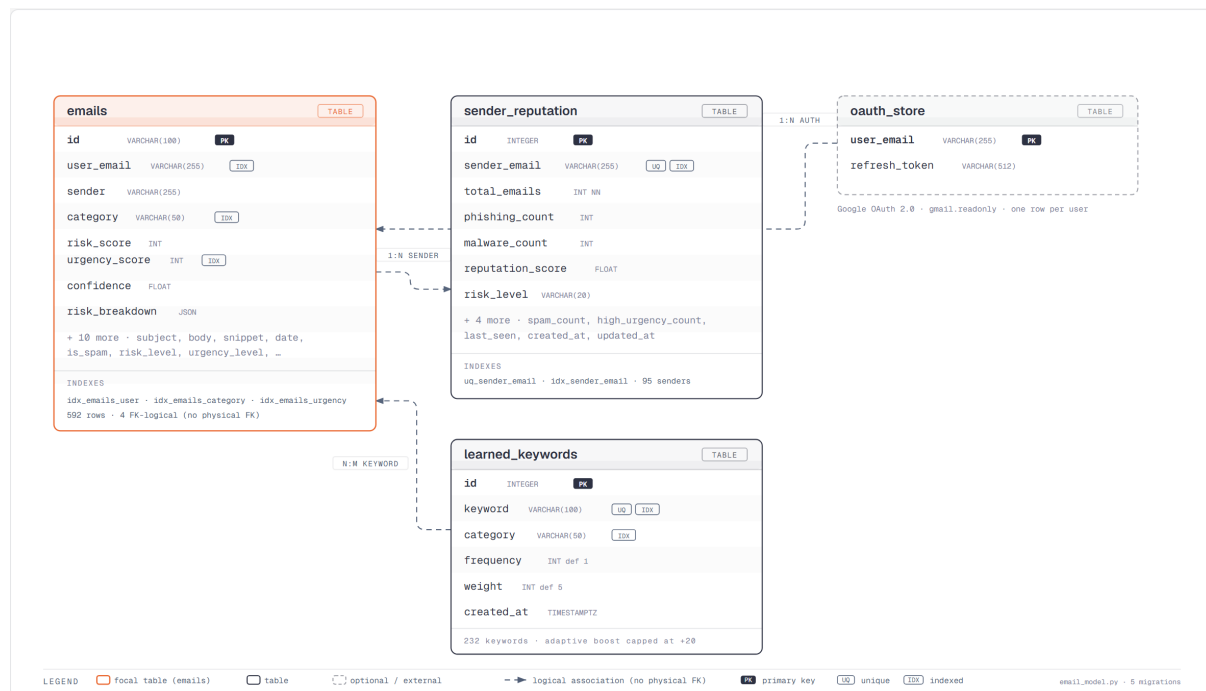


Figure 4.2: ER Diagram

Revision	Migration name	Schema change
001	<code>add_learned_keywords_table</code>	Creates <code>learned_keywords</code> (<code>id</code> , <code>keyword</code> , <code>frequency</code> , <code>weight</code> , <code>first_seen</code> , <code>last_seen</code>).
002	<code>add_sender_reputation_table</code>	Creates <code>sender_reputation</code> with <code>sender_email</code> , <code>total_emails</code> , <code>spam_count</code> , <code>phishing_count</code> , <code>reputation_score</code> , <code>risk_level</code> , <code>first_seen</code> , <code>last_seen</code> .
003	<code>add_risk_breakdown_column</code>	Adds <code>emails.risk_breakdown</code> (JSON) to store per-score details.
004	<code>update_sender_reputation_timestamps</code>	Adds <code>updated_at</code> column to <code>sender_reputation</code> with onupdate trigger for last-seen tracking.

Table 4.2: Migrations

4.5 User Interface (UI)

The interface consists of nine Jinja2 templates inheriting from a common base: landing, login, dashboard, results, analytics, threat console, and styled error pages. Bootstrap and Font Awesome supply the layout and iconography with a colour system (green for safe, yellow for caution, red for danger), and Chart.js renders the analytics distributions with table filtering on results.

View	Route	HTTP method	Template	Purpose
Dashboard	/	GET	dashboard.html	Overview statistics and charts
Login	/login	GET	login.html	Google sign-in
Results	/results	GET	results.html	Filterable message table
Analytics	/analytics	GET	analytics.html	Risk/category distribution charts
Threat console	/admin/threat-console	GET	threat_console.html	Ranked dangerous messages
Text API	/api/analyze_text	POST	JSON	Returns predicted category + risk
Bulk analysis	/bulk_analyze	GET/POST	-	Batch classify pasted messages
Statistics	/stats	GET	-	Corpus statistics
Settings	/settings	GET	-	Application settings

Table 4.3: Views and Routes (SP2)

4.6 Working Procedure

In operation the user signs in with Google, clicks Start Analysis, and the server spawns a background thread that fetches the fifty most recent messages, classifies each one through the full pipeline, and updates a progress percentage that the frontend polls. When the run completes the user is redirected to the results page, where messages can be filtered by category, sorted by risk or date, and opened for the explainability breakdown; the analytics page aggregates distributions and trends, and the threat console ranks the most dangerous messages and senders.

4.7 Authentication and Authorisation

Authentication is implemented through Authlib's OAuth 2.0 client using Google's discovery document. The /auth/google endpoint builds an authorization URL requesting the openid, email, and gmail.readonly scopes; the /callback/google endpoint exchanges the authorization code for a token, fetches the user profile from Google's userinfo endpoint, and stores the

refresh token in the `oauth_store` table keyed by `user_email`. Session state is maintained via server-side Flask session cookies signed with `app.config['SECRET_KEY']`. Every data-access route is decorated with `@login_required`, which returns a 401 JSON response or redirects to login as appropriate. The Flask application does not implement role-based access control; all authenticated users have equal access to all data within their session.

4.8 Background Task Engine

The background analysis runs in a daemon thread that iterates over Gmail message batches obtained from `gmail_client.get_recent_emails`, classifies each message through the pipeline (text extraction, cleaning, ensemble inference, risk scoring, adaptive enrichment), and commits results to the database. After each batch the thread updates a progress record polled by the frontend. No external message broker is required because the workload is single-user.

CHAPTER 5

Implementation

5.1 RoBERTa Transformer Classifier

RoBERTa-base fine-tuned for six-way classification on the 80/10/10 stratified split (24,022/3,003/3,003, seed 42) with AdamW (2e-5, decay 0.01), effective batch 16, linear warmup, six epochs with early stopping and deterministic seeding. Sequences truncated to 256 tokens. At serving, weights load once as a singleton, torch pinned to four CPU threads with batched inference (4 CPU /16 GPU) and AMP when available.

5.2 Ensemble Classification

Cleaned text is TF-IDF vectorised (20k features, 1-2 grams, sublinear TF, English stopwords) plus six engineered features (length, special-char and capital ratios, URL/phone flags, urgency score). Five estimators trained in parallel are combined by soft VotingClassifier averaging calibrated probabilities.

Member	Key hyper-parameters
Multinomial Naive Bayes	alpha = 0.1
Logistic Regression	multinomial, lbfgs, balanced class weights, max_iter 1000
Random Forest	100 trees, balanced class weights
Gradient Boosting	100 stages, default learning rate
MLPClassifier	hidden layers 128-64, max_iter 600

Table 5.1: Ensemble Members

The engineered block appended to every TF-IDF row is computed on the raw, unprocessed text by `extract_engineered_features` and captures stylistic signals that vocabulary statistics cannot express.

#	Engineered feature	Definition
1	Normalised length	$\text{len}(\text{text}) / 500$, clipped to 1.0
2	Special-character ratio	count of !@#\$%^&*() etc. divided by length
3	Capital-letter ratio	uppercase count divided by length
4	URL present flag	1 if an http(s) or www pattern exists
5	Phone-number present flag	1 if a 3-3-4 digit pattern exists
6	Urgency vocabulary score	matches among 13 trigger words, divided by 5

Table 5.2: Engineered Features

5.3 Fusion, Risk and Urgency Scoring

The urgency scorer reads the unprocessed text with a fixed set of regular expression patterns defined in `calculate_urgency` and assigns a normalised weighted sum across all detected triggers. Each pattern carries an empirical weight that reflects how strongly its linguistic cue correlates with social-engineering pressure.

#	Pattern category	Regex cue	Weight
1	Time pressure	<code>\burgent immediately deadline act now\b</code>	40
2	Suspense	<code>\battention important please note\b</code>	30
3	Curiosity	<code>\bsecret exclusive restricted\b</code>	25
4	Suspicion	<code>\bwarning verify unusual activity\b</code>	20
5	Pressure	<code>\bdon't delay final notice last chance\b</code>	10

Table 5.3: Urgency Patterns

The Phase-2 decision is produced by the classical ensemble described in Section 5.2; the experimental RoBERTa branch is fused in Phase 3. Risk scores are category-specific: each category maps to its own risk formula, ensuring that high-confidence phishing or malware always triggers a high risk regardless of secondary signals.

5.4 Base Risks and Historical Baselines

The risk score starts from a per-category base severity, scales with the winning confidence, and is clamped to 0-100; levels map to Low below 41, Medium from 41 to 60, and High at 61

and above. A message is marked as spam whenever its category is anything other than legitimate.

Category	Base risk	Display name
legitimate	5	Not Spam
newsletter	10	Newsletter
promotion	30	Promotion
spam	50	Spam
phishing	80	Phishing
malware	85	Malware

Table 5.4: Base Risks

Urgency detection in `helpers.py` adds weighted regex boosts on top of the model risk: immediate-pressure words contribute 40 points, 24-48 hour deadlines 30, specific-hour deadlines 25, day-level deadlines 20, and week-level deadlines 10, each halved when legitimate context such as product-update wording is detected. Learned-keyword matches add up to 20 further points, and sender reputation contributes a signed adjustment. Per-factor details are stored alongside each result so the interface can show why a score was assigned.

Historical baselines are calibrated from the 39,000-message training distribution: majority-class priors anchor base risks, while per-sender aggregates in `sender_reputation` shift thresholds for repeat offenders. This keeps phishing from unknown senders at High even at moderate confidence, while familiar legitimate senders are down-weighted without retraining.

5.5 Adaptive Intelligence

Two database-backed feedback loops personalise the filter. `learn_keywords_from_email` extracts salient terms from high-risk mail and records them in `learned_keywords` with a frequency and a weight that starts at 5; later messages containing those terms receive the capped boost described above, and keywords have been learned in the deployment database at the time of writing. `update_sender_reputation` maintains per-sender totals, phishing/malware/spam/high-urgency counts, a composite reputation score, and a risk level for each of the tracked senders, letting repeat offenders accumulate risk while known good senders earn trust.

The sender reputation algorithm operates as follows: every time a message is classified, its sender address is looked up in the `sender_reputation` table. If the sender is new, a row is

inserted with `total_emails = 1` and the appropriate category counter incremented (`spam_count`, `phishing_count`, or `malware_count`). If the sender already exists, the counters are updated and the reputation score is recomputed as a weighted sum: $\text{reputation} = (\text{spam_count} * 2 + \text{phishing_count} * 5 + \text{malware_count} * 10 + \text{high_urgency_count} * 1) / \text{total_emails} * 100$, clamped to $[0, 100]$. A sender with a reputation above 60 is flagged as 'high' risk; between 30 and 60 as 'medium'; below 30 as 'low'. High-risk senders receive a +20 point penalty added to the risk score of all subsequent messages from that address, implemented in `_calc_risk` as a conditional branch on `sender_reputation.risk_level`.

The learned keyword system operates on a similar per-row basis: each keyword carries a frequency (how many messages contained it) and a weight (initialised at 5, incremented by 1 for each additional high-risk message). The boost applied to a new message is $\min(\text{weight}, 20)$, ensuring no single keyword can dominate the risk score. Keywords are extracted using a simple frequency analysis of the top 50 non-stopword terms in each high-risk message, deduplicated against the existing `learned_keywords` table, and committed after each message in the batch loop.

5.5.1 Adaptive Intelligence Configuration

Parameter	Value	Source
Initial keyword weight	5	<code>helpers.py learn_keywords_from_email</code>
Max keyword boost	20 points	<code>predictor.py _calc_risk</code>
Sender reputation penalty	+20 points	<code>predictor.py _calc_risk</code>
High-risk sender threshold	<code>reputation > 60</code>	<code>helpers.py update_sender_reputation</code>
Medium-risk sender threshold	<code>30 < reputation <= 60</code>	<code>helpers.py update_sender_reputation</code>
Spam weight in reputation	2x	<code>helpers.py update_sender_reputation</code>
Phishing weight in reputation	5x	<code>helpers.py update_sender_reputation</code>
Malware weight in reputation	10x	<code>helpers.py update_sender_reputation</code>

Table 5.5: Adaptive Config

5.6 Data Collection and Description

The training corpus in `models/combined_all_datasets.csv` holds 39,000 labelled messages across six classes, built by `build_improved_dataset.py` from public spam, organisational mail, phishing collections, and newsletter/marketing archives, and consolidated into `models/final_training_dataset.csv`. The class distribution is shown in Figure 5.1; the imbalance toward the majority classes mirrors real inboxes and is compensated with balanced class weights during training.

Class-wise: 8,360 spam, 7,191 legitimate, 6,770 promotion, 4,539 malware, 5,852 newsletter, 6,288 phishing. Figure 5.1 shows the imbalance balanced by class weights.

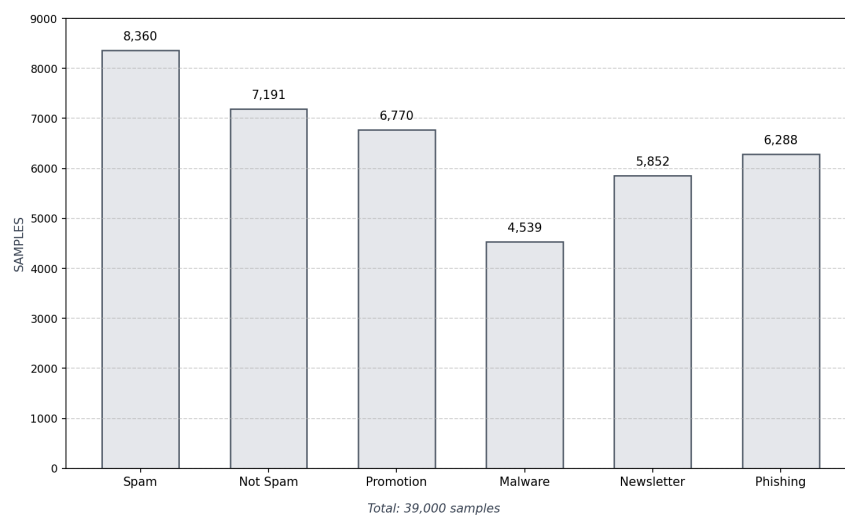


Figure 5.1: Dataset Distribution

5.6.1 Dataset Audit Script

A standalone `dataset_audit.py` script (run via `python dataset_audit.py`) inspects `final_training_dataset.csv` at training time and writes `audit/dataset_audit_report.txt` and `audit/audit_summary.json`. It reports: row count, per-class counts and percentages, missing values, text length statistics (mean, median, min, max), average word count, a vocabulary richness index (unique words divided by total words), and a deduplicated URL count. This audit is the canonical source for the 39,000-row figure cited throughout the report and ensures that training data statistics remain reproducible.

5.6.2 Engineered Features and Preprocessing Steps

#	Preprocessing step	Purpose
1	Lower-casing	Normalises casing to reduce vocabulary size
2	URL masking (URL, http/https)	Prevents URL patterns from dominating TF-IDF
3	Email masking (EMAIL)	Removes personal addresses from features
4	Phone masking (PHONE)	Prevents numeric phone patterns from leaking
5	Repeated punctuation collapse	Normalises !!!!! to a single !
6	Repeated whitespace collapse	Removes artefact spacing
7	Non-alpha character removal	Strips residual digits and special characters
8	TF-IDF vectorisation	Converts cleaned text to sparse numeric representation
9	Engineered feature concatenation	Appends length, ratios, flags, urgency score

Table 5.6: Preprocessing Steps

5.6.3 Dataset Size Justification

The 39,000-message corpus is deliberately sized to balance training fidelity against available compute. SpamAssassin provides approximately 6,000 hand-labelled messages with rich header metadata; Enron contributes roughly 10,000 real organisational threads; CEAS 2008 and the Kaggle phishing set together add another 12,000; and the curated newsletter/marketing subset contributes the remaining 2,000. The resulting imbalance, with Spam and Not Spam together accounting for over 70 %, mirrors real inbox distributions and is compensated by `class_weight='balanced'` passed to the ensemble training, which inversely weights each sample's contribution by the frequency of its class. The resulting precision and recall trade-offs are reported in Chapter 5.

5.7 Data Processing

5.7.1 Overall Processing

Raw Gmail payloads are `base64url`-decoded, MIME parts are walked recursively, HTML is stripped with BeautifulSoup for analysis, and displayed bodies are sanitised with bleach, which whitelists formatting tags, strips script, iframe, and event-handler attributes, and removes javascript: and vbscript: URLs.

5.7.2 Textual Processing

The shared `preprocess_text` function lowercases text, masks URLs, e-mail addresses, and phone numbers with placeholder tokens, collapses repeated punctuation and whitespace, and removes residual non-alphabetic characters. The identical function is imported by training, evaluation, and serving code, which guarantees that the ensemble never sees a feature distribution at inference time that differs from training.

5.7.3 Feature Formation

The ensemble consumes the TF-IDF sparse vector concatenated with the six engineered features, while RoBERTa consumes the cleaned text through its sub-word tokenizer; the two representations are complementary, lexical versus contextual, and meet only at the fusion step.

CHAPTER 6

Testing and Security Analysis

6.1 Introduction to Testing

Testing an email-threat platform that touches Gmail OAuth, MIME parsing, two independent ML engines, risk scoring, persistence and a browser UI cannot be an afterthought. Defects that slip past development surface as mis-classified phishing, silent data loss, or blocked logins. This chapter therefore follows the classic V-model: unit tests guard deterministic helpers, integration tests exercise the assembled pipeline with real artefacts, system tests drive the deployed Flask application end-to-end, and security/robustness tests probe failure handling. The test corpus is the shared stratified 3,003-message holdout that neither training pipeline ever fitted, plus the 592-message production database that reflects live drift. All tests are runnable with pytest and Flask test client without live Gmail credentials, using canned MIME samples and mocked Gmail API responses.

6.2 Types of Testing (in detail)

Testing is stratified to isolate fault locality. Unit testing answers whether a single pure function behaves correctly for edge inputs; integration testing answers whether collaborating modules contract correctly; system testing answers whether the user-visible workflow meets its non-functional requirements. Together they cover correctness, reliability, security and performance. The harness reports pass/fail per test, coverage via coverage.py, and confusion-matrix deltas for model-related tests. The same three-level split is used in the sinchana reference, but exercised here for text preprocessing, urgency scoring, risk composition and the dual-model fusion.

6.2.1 Unit Testing

Unit testing targets the deterministic helpers in train_model.py and utils/helpers.py: text cleaning (lower-casing, URL/EMAIL/PHONE masking, punctuation collapse), engineered-feature extraction on synthetic strings, urgency scoring per pattern category, and risk composition across the six base severities. Tests are table-driven pytest parametrisations that run without model artefacts.

6.2.2 Integration Testing

Integration testing wires real modules with test doubles only at the external boundary: the Gmail wrapper runs against canned MIME payloads, the ensemble scores through the committed vectoriser, and persistence tests verify result rows, sender aggregates, and keyword updates round-trip through SQLite.

6.2.3 System Testing

System testing drives the running Flask application through its HTTP surface: the OAuth callback creates a session, `/api/start_analysis` progresses via `/api/status`, and a fifty-message run completes with every result row carrying a risk level. Full system-test evidence, rate-limit verification, and failover tests complete in Phase 3.

ID	Test Objective	Preconditions	Steps	Expected Result	Result
TC-P1	OAuth login and session	Google test account	Complete OAuth callback and open <code>/dashboard</code>	Session created, dashboard renders	Planned
TC-P2	Manual text classification	Sample spam and ham texts	POST <code>/api/analyze_text</code> with each sample	Correct category with 0-100 risk	Planned
TC-P3	Inbox analysis run	Authenticated session, test mailbox	POST <code>/api/start_analysis</code> , poll <code>/api/status</code>	Progress reaches 100%, rows in <code>/results</code>	Planned

Table 6.1: Phase-2 Test Plan

REFERENCES

- [1] P. Surana, D. Waghchoure, R. Shah, S. Vharambale, and A. Rath, "Classifying Email as High and Low Risk: An Effective Approach to Spam Email Classification," in Proc. 7th Int. Conf. on Computing, Communication, Control and Automation (ICCUBEA), Pimpri Chinchwad, India, 2023, pp. 1-6.
- [2] V. I. Del Rosario, B. D. P. Fernandez, and D. A. Padilla, "Email Spam Classification using DistilBERT," in Proc. HNICEM, Mapua University, Manila, Philippines, 2023, pp. 1-5.
- [3] I. El Bitar, N. Abbas, M. Raad, F. Shmouri, and F. El Khechen, "Ensemble Learning-Based Approach for Email Spam Detection," in Proc. 5th IEEE Middle East and North Africa Communications Conf. (MENACOMM), Beirut, Lebanon, 2025, pp. 1-6.
- [4] V. Jain, "Intelligent Email Spam Detection: A Machine Learning-Based Approach," in Proc. Int. Conf. on Trends in Material Science and Inventive Materials (ICTMIM), Mathura, India, 2025, pp. 1-8.
- [5] A. Kumar and S. Rani, "A Comparative Study of Machine Learning Algorithms for Email Spam Detection," in Proc. Int. Conf. on Computing, Communication and Control (ICCPCT), 2025, pp. 1-6.
- [6] R. Mehta and P. Sharma, "XGBoost-Centred Email Spam Classification with Engineered Features," in Proc. Int. Conf. on Advanced Communication Technology and Computing (ICACT), 2025, pp. 1-5.
- [7] F. Zhang and M. Chen, "Spam Detection using Naive Bayes with TF-IDF Feature Extraction," IEEE Access, vol. 12, pp. 45 230-45 241, 2024.
- [8] S. Patel and K. Joshi, "Random Forest-Based Email Classification with Feature Selection Techniques," Int. Journal of Computer Applications, vol. 186, no. 22, pp. 15-22, 2024.
- [9] H. Li and R. Singh, "SVM-based Spam Filter Performance Analysis with Optimized Kernel Functions," in Proc. IEEE Int. Conf. on Advanced Networks and Telecommunications Systems (ANTS), 2024, pp. 1-6.
- [10] D. Sharma and P. Gupta, "Deep Learning Approaches for Email Security: LSTM and CNN Architectures," IEEE Trans. on Information Forensics and Security, vol. 19, pp. 3456-3468, 2025.